

TestDoc Agent

**Agente Inteligente para Geração e Avaliação de Documentação de
Testes por Feature**

Eduardo Barboza

Maio de 2026

Versão atualizada com Few-shot Prompting

Documentação do Projeto - TestDoc Agent

1. Título do Projeto

TestDoc Agent: Agente Inteligente para Geração e Avaliação de Documentação de Testes por Feature.

2. Resumo

O projeto propõe o desenvolvimento de um agente inteligente voltado para auxiliar desenvolvedores na definição de estratégias de teste para novas funcionalidades de software. A partir da descrição de uma feature, issue, pull request ou conjunto de regras de negócio, o agente será capaz de identificar riscos, recomendar tipos de teste, priorizar cenários críticos e gerar uma documentação estruturada para apoiar o processo de desenvolvimento e validação.

A solução não tem como foco inicial gerar automaticamente o código dos testes, mas sim produzir uma documentação técnica que oriente quais testes devem ser implementados, por que eles são necessários e quais cenários devem ser priorizados. Para isso, o agente utilizará padrões de agentes como Planning, Tool Use, Retrieval-Augmented Generation e Reflection/Critic.

Além desses padrões, o projeto adotará Few-shot Prompting como técnica de aprendizagem contextual. O agente será orientado por exemplos previamente rotulados de funcionalidades e estratégias de teste esperadas, permitindo aprender o formato, o nível de detalhe e os critérios esperados para a documentação gerada, sem necessidade de fine-tuning no MVP.

A validação será realizada por meio de uma base controlada de funcionalidades, acompanhada de gabaritos esperados. A saída do agente será comparada com esses gabaritos usando métricas objetivas como precisão, recall, F1-score, acurácia da classificação de criticidade, taxa de sugestões inválidas e pontuação por rubrica.

3. Problema

Durante o desenvolvimento de software, é comum que funcionalidades sejam implementadas sem uma estratégia clara de testes. Muitas vezes, os desenvolvedores sabem o que foi alterado no código, mas não documentam adequadamente quais tipos de testes são necessários para validar a feature.

Isso pode gerar problemas como:

- ausência de cobertura para regras de negócio críticas;
- testes focados apenas no caminho feliz;
- falta de validação de cenários de erro;
- baixa padronização entre diferentes desenvolvedores;
- dificuldade de revisar pull requests;
- dependência excessiva de QA ou revisão humana para identificar lacunas de teste;
- documentação insuficiente sobre o motivo de cada teste existir.

Diante disso, surge a necessidade de uma ferramenta que auxilie o desenvolvedor a transformar a descrição de uma feature em uma estratégia de testes documentada, organizada e justificável.

4. Objetivo Geral

Desenvolver um agente inteligente capaz de gerar documentação de testes por feature, recomendando tipos de teste, riscos, cenários prioritários e critérios de validação com base na descrição funcional e técnica de uma mudança de software.

5. Objetivos Específicos

- Identificar automaticamente os riscos associados a uma funcionalidade.
- Classificar a criticidade da funcionalidade como baixa, média, alta ou crítica.

- Recomendar tipos de teste adequados, como unitário, integração, E2E, segurança, contrato/API, performance ou regressão.
- Gerar cenários de teste esperados, incluindo casos positivos, negativos e de borda.
- Priorizar os testes mais importantes conforme risco e impacto.
- Produzir uma documentação padronizada para apoiar desenvolvedores e revisores.
- Utilizar Few-shot Prompting para orientar o agente por meio de exemplos rotulados.
- Validar a qualidade das recomendações por meio de gabaritos e métricas objetivas.
- Comparar o desempenho do agente com uma baseline simples.

6. Escopo do Projeto

6.1 Dentro do escopo

O projeto contemplará:

- entrada de dados sobre uma funcionalidade;
- análise da descrição da feature;
- identificação de regras de negócio e dependências;
- recomendação de tipos de teste;
- geração de documentação técnica;
- uso de exemplos rotulados via Few-shot Prompting;
- criação de uma base controlada de funcionalidades;
- criação de gabaritos esperados;
- avaliação quantitativa da saída do agente.

6.2 Fora do escopo inicial

O MVP não terá como prioridade:

- geração automática completa de código de teste;
- integração obrigatória com repositórios reais;
- execução real de testes automatizados;
- fine-tuning ou treinamento supervisionado com ajuste de pesos do modelo;
- substituição de um time de QA;
- validação manual obrigatória por especialistas.

Essas funcionalidades podem ser consideradas em versões futuras.

7. Usuário-Alvo

O usuário principal do sistema será o desenvolvedor de software que precisa documentar ou planejar testes para uma feature antes, durante ou após a implementação.

Usuários secundários:

- revisores de pull request;
- líderes técnicos;
- analistas de QA;
- equipes acadêmicas ou de pesquisa em engenharia de software;
- estudantes de desenvolvimento de software.

8. Entrada do Agente

O agente poderá receber uma descrição textual da funcionalidade contendo informações como:

- nome da funcionalidade;
- descrição geral;
- regras de negócio;

- entradas esperadas;
- saídas esperadas;
- dependências técnicas;
- integrações externas;
- possíveis falhas conhecidas;
- nível de criticidade esperado, quando informado;
- arquivos ou módulos impactados, quando disponíveis.

Exemplo de entrada

Funcionalidade: Checkout com cupom

Regras:

- Cupom pode estar ativo, expirado ou esgotado.
- Cupom não pode ser usado duas vezes pelo mesmo usuário.
- Valor final não pode ficar negativo.
- Pagamento só deve ser criado se o pedido for válido.

9. Saída Esperada do Agente

A saída será uma documentação estruturada de testes por feature.

Modelo de saída

Funcionalidade: Checkout com cupom

Resumo:

Funcionalidade responsável por aplicar cupons de desconto durante o processo de checkout.

Criticidade:

Crítica

Riscos identificados:

1. Aceitar cupom expirado.
2. Aceitar cupom esgotado.
3. Permitir reuso de cupom pelo mesmo usuário.
4. Gerar valor final negativo.
5. Criar pagamento para pedido inválido.

Tipos de teste recomendados:

- Unitário
- Integração
- E2E
- Segurança/regra de negócio

Cenários prioritários:

1. Validar cálculo de desconto para cupom ativo.
2. Rejeitar cupom expirado.
3. Rejeitar cupom esgotado.
4. Impedir reutilização do cupom.
5. Impedir valor final negativo.
6. Criar pagamento apenas para pedido válido.

Justificativa:

A funcionalidade envolve regra de negócio financeira, validação de estado do cupom, criação de pedido e integração com pagamento, portanto exige testes unitários, de integração e de fluxo completo.

10. Design Patterns de Agentes Utilizados

O projeto utilizará pelo menos três design patterns de agentes. A proposta recomenda quatro patterns principais. Além deles, o projeto inclui Few-shot Prompting como técnica de aprendizagem contextual, detalhada na Seção 11.

10.1 Planning Pattern

O agente deve decompor a análise da feature em etapas antes de gerar a documentação.

Aplicação no projeto:

1. Entender a funcionalidade.
2. Identificar regras de negócio.
3. Detectar dependências técnicas.
4. Mapear riscos.
5. Classificar criticidade.
6. Recomendar tipos de teste.
7. Priorizar cenários.
8. Gerar documentação final.

Justificativa: esse padrão reduz respostas genéricas e obriga o agente a analisar a feature por partes, aumentando a chance de identificar riscos relevantes.

10.2 Retrieval-Augmented Generation Pattern

O agente poderá consultar uma base de conhecimento com exemplos de funcionalidades, tipos de teste, riscos conhecidos e gabaritos anteriores.

A base de conhecimento poderá conter:

- matriz de decisão de tipos de teste;
- exemplos de funcionalidades e riscos esperados;
- documentação de QA;
- checklists de teste;
- gabaritos criados para avaliação;
- exemplos de bugs e falhas comuns.

Justificativa: esse padrão permite que o agente gere recomendações baseadas em exemplos e critérios previamente definidos, em vez de depender apenas da geração livre do modelo.

10.3 Tool Use Pattern

O agente poderá usar ferramentas auxiliares para consultar informações, validar saídas ou calcular métricas.

Ferramentas possíveis:

- parser de entrada da feature;
- classificador de risco;
- calculadora de métricas;
- comparador entre saída do agente e gabarito;
- leitor de arquivos de projeto ou PR, em versões futuras;
- ferramenta para geração automática do relatório final.

Justificativa: esse padrão permite que o agente não apenas gere texto, mas também execute operações objetivas, como comparar listas, calcular precisão e recall, ou verificar se tipos de teste esperados foram recomendados.

10.4 Reflection / Critic Pattern

Após gerar uma primeira versão da documentação, o agente fará uma revisão crítica da própria resposta.

O agente avaliador verificará:

- se algum risco óbvio foi omitido;
- se algum tipo de teste recomendado é irrelevante;
- se a criticidade está coerente;
- se os cenários incluem casos positivos, negativos e de borda;
- se há justificativa para cada tipo de teste;
- se a documentação está completa.

Justificativa: esse padrão melhora a qualidade da resposta final e reduz omissões, exageros ou sugestões genéricas.

11. Técnica de Aprendizagem e Orientação do Agente

O projeto não utilizará, em sua versão inicial, treinamento supervisionado com ajuste de pesos do modelo, como fine-tuning. Em vez disso, será adotada uma abordagem de aprendizagem contextual, na qual o agente é orientado por exemplos, critérios e regras fornecidos no contexto da execução.

A estratégia principal será composta por:

- RAG, para recuperar critérios, matriz de decisão, exemplos anteriores e documentação de apoio;
- Few-shot Prompting, para apresentar ao agente exemplos de funcionalidades já avaliadas e suas respectivas estratégias de teste esperadas;
- Matriz de decisão, para associar características da feature aos tipos de teste mais adequados;
- Rubrica de avaliação, para medir objetivamente a qualidade da saída;
- Baseline fixa, utilizada apenas como método comparativo.

11.1 Few-shot Prompting

O Few-shot Prompting será utilizado para fornecer ao agente exemplos previamente rotulados de entrada e saída esperada. Esses exemplos funcionam como demonstrações de como o agente deve interpretar uma feature, identificar riscos, escolher tipos de teste e justificar suas recomendações.

Dessa forma, o agente não parte apenas de uma instrução genérica. Ele recebe exemplos concretos de funcionalidades já analisadas, com gabarito esperado, e usa esse padrão para responder a novas funcionalidades.

11.2 Exemplo de Few-shot

Exemplo fornecido ao agente:

Entrada:

Funcionalidade: Login com 2FA

Regras:

- Código expira em 5 minutos.
- Código não pode ser reutilizado.
- Usuário bloqueado não pode autenticar.

Saída esperada:

Riscos:

- Código expirado aceito.
- Código reutilizado.
- Brute force no código.

- Login de usuário bloqueado.

Tipos de teste:

- Unitário.
- Integração.
- E2E.
- Segurança.

Criticidade:

Alta.

Justificativa:

A feature envolve autenticação, validação temporal, controle de acesso e risco de abuso, exigindo validações funcionais e de segurança.

11.3 Papel do Few-shot no projeto

O Few-shot Prompting será usado para orientar três aspectos principais da saída do agente:

- formato da documentação gerada;
- nível de detalhe esperado na identificação de riscos;
- critérios para escolha dos tipos de teste e da criticidade.

Assim, o Few-shot Prompting complementa o RAG. Enquanto o RAG recupera informações relevantes da base de conhecimento, o Few-shot mostra ao agente como aplicar essas informações na prática.

11.4 Diferença entre Few-shot, RAG e baseline

Elemento	Função no projeto
Few-shot Prompting	Ensina por exemplos no contexto, mostrando entradas e saídas esperadas.
RAG	Recupera conhecimento, regras, matriz de decisão e exemplos relevantes.
Baseline	Serve apenas como comparação, não como método de treinamento.
Fine-tuning	Não será usado no MVP; pode ser uma evolução futura.

12. Arquitetura Conceitual do Agente

O sistema pode ser organizado em módulos ou agentes especializados.

```
Entrada da Feature
↓
PlannerAgent
↓
RAG/FewShotContextBuilder
↓
RiskAnalyzerAgent
↓
TestStrategyAgent
↓
DocumentationAgent
↓
EvaluatorAgent
↓
ReflectionAgent
↓
Documento Final de Testes
```

12.1 PlannerAgent

Responsável por organizar a análise da funcionalidade em etapas.

12.2 RAG/FewShotContextBuilder

Responsável por recuperar exemplos, critérios, matriz de decisão e exemplos few-shot que serão inseridos no contexto do agente.

12.3 RiskAnalyzerAgent

Responsável por identificar riscos técnicos, funcionais e de negócio.

12.4 TestStrategyAgent

Responsável por associar riscos e características da feature aos tipos de teste adequados.

12.5 DocumentationAgent

Responsável por gerar a documentação final em formato padronizado.

12.6 EvaluatorAgent

Responsável por comparar a saída do agente com gabaritos ou checklists.

12.7 ReflectionAgent

Responsável por revisar a documentação antes da entrega final.

13. Tipos de Teste Considerados

Tipo de teste	Quando recomendar
Unitário	Quando houver regras de negócio isoladas, cálculos, validações ou funções específicas.
Integração	Quando houver comunicação entre módulos, banco de dados, serviços ou APIs internas.
E2E	Quando a feature envolver fluxo completo do usuário.
Segurança	Quando houver autenticação, autorização, permissões, dados sensíveis ou risco de abuso.
Contrato/API	Quando houver integração entre serviços ou exposição de endpoints.
Performance	Quando houver consultas pesadas, relatórios, alto volume de dados ou tempo de resposta crítico.
Regressão	Quando a mudança impactar funcionalidades existentes.
UI/Componente	Quando houver alteração de interface, formulário ou comportamento visual.
Migração/Banco	Quando houver alteração em schema, migrations ou persistência crítica.

14. Matriz de Decisão Inicial

A matriz de decisão servirá como apoio para orientar o agente e também como parte da validação.

Característica da feature	Testes esperados
Altera regra de negócio isolada	Unitário
Altera endpoint/API	Integração e contrato/API
Altera fluxo completo do usuário	E2E
Envolve autenticação	Segurança, integração e E2E
Envolve autorização ou permissões	Segurança e casos negativos
Envolve banco de dados	Integração e migração

Envolve serviço externo	Integração com mock e contrato/API
Envolve cálculo financeiro	Unitário, integração e casos de borda
Envolve upload de documento	Integração, segurança e validação de arquivo
Envolve relatório pesado	Performance e integração
Envolve exclusão de dados	Segurança, integração e regressão

15. Base de Avaliação

Para validar o agente, será criada uma base controlada com aproximadamente 10 a 20 funcionalidades.

Exemplos de funcionalidades

1. Login com 2FA.
2. Cadastro de usuário.
3. Recuperação de senha.
4. Checkout com cupom.
5. Upload de documento.
6. Emissão de certificado.
7. Alteração de permissões.
8. Exclusão de conta.
9. Integração com API externa.
10. Geração de relatório financeiro.

Para cada funcionalidade, serão documentados:

- descrição da funcionalidade;
- regras de negócio;
- entradas e saídas esperadas;
- dependências técnicas;
- possíveis falhas conhecidas;
- criticidade esperada;
- exemplos de saída esperada para uso em few-shot, quando aplicável.

Essa base será usada tanto como conjunto de avaliação quanto como fonte de exemplos para orientar o agente em execuções futuras.

16. Gabarito Esperado

Para cada funcionalidade, será criado um gabarito contendo a estratégia considerada correta.

Exemplo: Login com 2FA

Riscos esperados:

- Código expirado ser aceito.
- Código reutilizado ser aceito.
- Ataques de brute force no código.
- Login de usuário bloqueado.
- Falha no envio do código.

Tipos de teste esperados:

- Unitário.
- Integração.

- E2E.
- Segurança.

Prioridade esperada:

- Alta: expiração, reutilização, brute force e usuário bloqueado.
- Média: falha no envio de e-mail ou SMS.

17. Método de Avaliação

A avaliação será feita comparando a documentação gerada pelo agente com o gabarito esperado.

A saída do agente será transformada em uma lista de itens avaliáveis:

- riscos identificados;
- tipos de teste recomendados;
- cenários sugeridos;
- criticidade atribuída;
- prioridades definidas;
- sugestões inválidas ou irrelevantes.

Também será possível avaliar o efeito do Few-shot Prompting comparando duas configurações do agente: uma sem exemplos few-shot e outra com exemplos few-shot. Essa comparação permitirá verificar se os exemplos rotulados melhoram a precisão, o recall ou reduzem sugestões inválidas.

18. Métricas de Avaliação

18.1 Precisão

Mede quantas sugestões feitas pelo agente estavam corretas.

Precisão = sugestões corretas / total de sugestões feitas

Exemplo:

Agente sugeriu 10 riscos.

7 estavam corretos.

Precisão = $7/10 = 70\%$

18.2 Recall / Cobertura

Mede quantos itens esperados o agente conseguiu encontrar.

Recall = itens corretos encontrados / total de itens esperados

Exemplo:

Gabarito tinha 8 riscos esperados.

Agente encontrou 6.

Recall = $6/8 = 75\%$

18.3 F1-score

Combina precisão e recall em uma única métrica.

$F1 = 2 \times (\text{Precisão} \times \text{Recall}) / (\text{Precisão} + \text{Recall})$

Essa métrica pode ser aplicada para riscos identificados, tipos de teste sugeridos, cenários recomendados e prioridades atribuídas.

18.4 Acurácia da Criticidade

Avalia se o agente classificou corretamente a criticidade da funcionalidade.

Acurácia = classificações corretas / total de funcionalidades

Funcionalidade	Gabarito	Agente	Acertou?
Login com 2FA	Alta	Alta	Sim
Página institucional	Baixa	Média	Não
Checkout	Crítica	Crítica	Sim
Upload de documento	Alta	Média	Não

18.5 Pontuação de Priorização

Avalia se o agente atribuiu prioridades coerentes aos testes e riscos.

Situação	Pontuação
Prioridade correta	1 ponto
Prioridade próxima	0,5 ponto
Prioridade errada	0 ponto

18.6 Taxa de Sugestões Inválidas

Mede o quanto o agente gera recomendações irrelevantes.

Taxa de sugestões inválidas = sugestões inválidas / total de sugestões

Exemplos de sugestões inválidas:

- sugerir teste de performance para uma tela estática simples;
- sugerir teste de pagamento para uma funcionalidade de perfil;
- sugerir teste de segurança sem relação com o requisito;
- recomendar E2E para uma função interna simples sem fluxo de usuário.

Quanto menor essa taxa, melhor.

19. Rubrica de Avaliação

Cada funcionalidade poderá receber uma nota de 0 a 100.

Dimensão	Peso
Identificação correta dos riscos	30 pontos
Escolha adequada dos tipos de teste	25 pontos
Priorização correta	20 pontos
Cobertura de casos críticos	15 pontos
Baixa taxa de sugestões irrelevantes	10 pontos

Funcionalidade: Login com 2FA
Pontuação final: 82/100

20. Baseline de Comparação

Para demonstrar que o agente é melhor do que uma abordagem genérica, será criada uma baseline simples.

20.1 Baseline proposta

Para toda funcionalidade, recomendar sempre:

- teste unitário;
- teste de integração;
- teste E2E.

Depois, os resultados da baseline serão comparados com os resultados do agente.

20.2 Exemplo de comparação

Método	Precisão	Recall	F1	Sugestões inválidas
Baseline fixa	45%	80%	57%	40%
Agente proposto com RAG + Few-shot	78%	75%	76%	12%

Essa comparação ajuda a demonstrar se o agente realmente gera recomendações mais precisas e contextualizadas.

21. Exemplo Completo de Avaliação

21.1 Entrada

Funcionalidade: Checkout com cupom

Regras:

- Cupom pode estar ativo, expirado ou esgotado.
- Cupom não pode ser usado duas vezes pelo mesmo usuário.
- Valor final não pode ficar negativo.
- Pagamento só deve ser criado se o pedido for válido.

21.2 Gabarito

Riscos esperados:

1. Aceitar cupom expirado.
2. Aceitar cupom esgotado.
3. Permitir reuso de cupom.
4. Gerar valor negativo.
5. Criar pagamento para pedido inválido.

Tipos esperados:

- Unitário para cálculo de desconto.
- Integração para cupom e pedido.
- Integração para pedido e pagamento.
- E2E para fluxo de checkout.
- Segurança/regra de negócio para reuso de cupom.

21.3 Saída hipotética do agente

Riscos:

1. Aceitar cupom expirado.
2. Gerar valor negativo.
3. Falha no pagamento.
4. Lentidão no checkout.

Tipos:

- Unitário.
- Integração.
- Performance.
- E2E.

21.4 Avaliação

Riscos corretos encontrados: 2 de 5
Riscos extras corretos/parciais: 1
Risco irrelevante/parcial: 1

Recall de riscos: $2/5 = 40\%$
Precisão de riscos: $3/4 = 75\%$

Tipos corretos encontrados: 3 de 5
Recall de tipos: 60%

22. Fluxo Geral do Projeto

1. Criar base de funcionalidades.
2. Criar gabarito esperado para cada funcionalidade.
3. Selecionar exemplos few-shot representativos.
4. Enviar cada funcionalidade ao agente.
5. Gerar documentação de testes por feature.
6. Comparar saída do agente com o gabarito.
7. Calcular precisão, recall, F1-score, acurácia e taxa de sugestões inválidas.
8. Comparar o agente com uma baseline genérica.
9. Comparar o agente com e sem Few-shot Prompting, se possível.
10. Analisar os resultados.

23. Possível Implementação do MVP

23.1 Tecnologias sugeridas

- Backend: Python com FastAPI ou Flask.
- LLM: API de modelo de linguagem.
- Base de conhecimento: arquivos JSON, CSV ou banco vetorial simples.
- Few-shot examples: conjunto de exemplos rotulados em JSON ou Markdown.
- Frontend opcional: React, Vite ou Next.js.
- Avaliação: scripts Python para cálculo das métricas.

23.2 Estrutura de dados sugerida

Feature:

```
{
  "id": "feature_001",
  "name": "Login com 2FA",
  "description": "Usuário realiza login e confirma acesso com código temporário.",
  "business_rules": [
    "Código deve expirar em 5 minutos",
    "Código não pode ser reutilizado",
    "Usuário bloqueado não pode autenticar"
  ],
  "dependencies": [
    "UserService",
    "AuthService",
    "EmailService"
  ],
}
```

```
"expected_criticality": "Alta"
}
```

Gabarito:

```
{
  "feature_id": "feature_001",
  "expected_risks": [
    "Código expirado aceito",
    "Código reutilizado",
    "Brute force no código",
    "Login de usuário bloqueado",
    "Falha no envio do código"
  ],
  "expected_test_types": [
    "unitário",
    "integração",
    "E2E",
    "segurança"
  ],
  "expected_criticality": "Alta"
}
```

Exemplo few-shot:

```
{
  "input_feature_id": "feature_001",
  "expected_output_example": {
    "identified_risks": [
      "Código expirado aceito",
      "Código reutilizado",
      "Brute force no código"
    ],
    "recommended_test_types": [
      "unitário",
      "integração",
      "E2E",
      "segurança"
    ],
    "criticality": "Alta",
    "justification": "A feature envolve autenticação e controle de acesso."
  }
}
```

24. Critérios de Sucesso

O projeto será considerado bem-sucedido se:

- o agente gerar documentação estruturada e compreensível;
- a saída incluir riscos, tipos de teste, prioridades e justificativas;
- o agente superar a baseline fixa em F1-score ou precisão;
- a taxa de sugestões inválidas for menor do que a baseline;
- o uso de Few-shot Prompting demonstrar melhora ou maior consistência na saída;
- a documentação gerada for útil para orientar a implementação de testes;
- o método de avaliação permitir mensurar objetivamente os acertos e erros.

25. Contribuição Esperada

A contribuição do projeto está em propor um agente que não apenas sugere testes de forma genérica, mas gera uma estratégia documentada de testes por funcionalidade, com justificativa, priorização e validação objetiva.

O projeto também contribui ao apresentar um método de avaliação aplicável a agentes voltados para apoio ao desenvolvimento de software, reduzindo a dependência exclusiva de avaliação subjetiva por especialistas.

A inclusão do Few-shot Prompting fortalece a proposta por permitir que o agente seja orientado por exemplos rotulados, tornando a geração mais consistente sem exigir treinamento supervisionado do modelo.

26. Definição Final do Projeto

O projeto consiste em desenvolver um agente inteligente chamado TestDoc Agent, cujo objetivo é auxiliar desenvolvedores na geração de documentação de testes por feature.

A partir da descrição de uma funcionalidade, o agente deverá:

1. entender o contexto da mudança;
2. recuperar exemplos e critérios relevantes via RAG;
3. usar exemplos few-shot para orientar o formato e o nível de detalhe da resposta;
4. identificar riscos funcionais, técnicos e de negócio;
5. recomendar os tipos de teste mais adequados;
6. sugerir cenários positivos, negativos e de borda;
7. classificar a criticidade da feature;
8. priorizar os testes mais importantes;
9. gerar uma documentação estruturada;
10. revisar a própria saída;
11. permitir avaliação objetiva por comparação com gabaritos.

A validação será feita com uma base de funcionalidades e gabaritos previamente definidos, usando métricas como precisão, recall, F1-score, acurácia, taxa de sugestões inválidas e comparação com uma baseline fixa.

27. Formulação Acadêmica Sugerida

27.1 Tema

Agentes inteligentes aplicados à documentação de estratégias de teste em desenvolvimento de software.

27.2 Problema de pesquisa

Como um agente inteligente pode auxiliar desenvolvedores na geração de documentação de testes por feature e como suas recomendações podem ser avaliadas objetivamente sem depender exclusivamente de especialistas de QA?

27.3 Hipótese

Um agente baseado em padrões como Planning, Retrieval-Augmented Generation, Tool Use e Reflection, combinado com Few-shot Prompting, pode gerar recomendações de teste mais contextualizadas do que uma estratégia genérica fixa, apresentando maior precisão, melhor F1-score e menor taxa de sugestões irrelevantes.

27.4 Objetivo geral

Desenvolver e avaliar um agente inteligente para geração de documentação de estratégias de teste por funcionalidade.

27.5 Método de avaliação

A avaliação será conduzida por meio da comparação entre a saída do agente e uma matriz de referência definida para um conjunto de funcionalidades. Serão utilizadas métricas quantitativas como precisão, recall, F1-score, acurácia da criticidade e taxa de sugestões inválidas. Também poderá ser feita uma comparação entre versões do agente com e sem Few-shot Prompting para analisar o impacto da técnica na qualidade das recomendações.

28. Entregáveis do Projeto

- Documento de definição do projeto.
- Base com 10 a 20 funcionalidades.
- Gabarito esperado para cada funcionalidade.
- Conjunto de exemplos few-shot rotulados.
- Protótipo do agente.
- Módulo de geração de documentação.
- Módulo de avaliação automática.
- Relatório com métricas de desempenho.
- Comparação com baseline fixa.
- Comparação opcional com e sem Few-shot Prompting.
- Conclusão sobre a efetividade do agente.

29. Próximos Passos

1. Definir as 10 primeiras funcionalidades da base de avaliação.
2. Criar o modelo JSON de entrada das funcionalidades.
3. Criar o modelo JSON dos gabaritos.
4. Selecionar exemplos para Few-shot Prompting.
5. Especificar o prompt principal do agente.
6. Implementar o primeiro protótipo.
7. Rodar o agente nas funcionalidades.
8. Calcular as métricas.
9. Comparar com a baseline.
10. Comparar a saída com e sem Few-shot Prompting, se viável.
11. Ajustar o agente com base nos erros encontrados.
12. Gerar o relatório final do projeto.